Data Quality Scoring Dashboard: An Automated Domain-Aware Profiling, Longitudinal Drift Analysis, and Interactive Remediation Framework

Prathusha
Faculty Project Supervisor, Dept. of CSE
Koneru Lakshmaiah Education Foundation
Hyderabad, Telangana, India
Assistant Professor

P. Poojith Varma Sai
Dept. of Computer Science & Engineering
Koneru Lakshmaiah Education Foundation
Hyderabad, Telangana, India
2420030749@klh.edu.in

V. Harshith Chandra Varma
Dept. of Computer Science & Engineering
Koneru Lakshmaiah Education Foundation
Hyderabad, Telangana, India
2420030020@klh.edu.in

V. Venkataakhilesh
Dept. of Computer Science & Engineering
Koneru Lakshmaiah Education Foundation
Hyderabad, Telangana, India
2420030705@klh.edu.in

***Abstract*—Modern enterprise decision-support systems and machine learning workflows ingest massive volumes of tabular data from heterogeneous, distributed sources. However, real-world data pipelines routinely suffer from silent data degradation, including high missingness ratios, record duplication, syntactic formatting violations, schema mutations, and extreme statistical anomalies. These defects induce the "Garbage In, Garbage Out" (GIGO) paradox, precipitating severe compounding downstream errors known as data cascades. Existing data verification frameworks frequently require complex domain-specific assertion scripting, impose heavy distributed runtime dependencies, lack domain-contextualized scoring, or fail to provide human-in-the-loop interactive remediation.**

This paper presents the Data Quality Scoring Dashboard, a full-stack, modular, automated framework designed for domain-aware tabular data profiling, longitudinal drift tracking, and prescriptive interactive remediation. The core engine formalizes a deterministic multi-criteria scoring model that quantifies four foundational quality dimensions: Completeness (S_C), Uniqueness (S_U), Consistency (S_K), and Validity (S_V). By incorporating domain-tailored weight matrices spanning General ($30/25/25/20$), Healthcare ($45/20/20/15$), Finance ($10/40/30/20$), and E-Commerce ($25/35/20/20$) presets, the platform maps multi-dimensional quality metrics into an actionable 0–100 composite health index (S_{overall}) and a four-tier classification taxonomy (Tier 1: Excellent to Tier 4: Poor). An integrated dual-snapshot comparator identifies longitudinal schema mutations (added, dropped, and type-altered attributes) alongside column-level missingness drift. Furthermore, an interactive "What-If" remediation sandbox simulates real-time composite score recovery prior to executing physical transformations, including deduplication, statistical imputation (mean, median, mode, constant), whitespace trimming, and Interquartile Range ($1.5 \times$ IQR) boundary clipping.

Empirical verification confirms a 100% pass rate across the automated verification test suite. Comprehensive evaluation across synthetic multi-domain benchmarks demonstrates significant composite quality recovery: General Domain increases from 93.47% to 99.72% ($+6.25\%$), Healthcare from 94.81% to 100.0% ($+5.19\%$), Finance from 93.70% to 99.67% ($+5.97\%$), and E-Commerce from 93.32% to 100.0% ($+6.68\%$). Computational scalability benchmarks demonstrate linear runtime scaling from $N = 10^2$ to $N = 10^5$ records, sustaining a peak processing throughput exceeding 42,000 rows per second on standard commodity hardware. The proposed framework establishes a transparent, interpretable, and computationally efficient paradigm for proactive tabular data observability.

***Index Terms*—Data quality assessment, tabular data profiling, completeness, data consistency, duplicate detection, quality scoring, statistical anomaly detection, interactive remediation, temporal drift monitoring, data observability.**

I. Introduction

In contemporary data science, business intelligence, and artificial intelligence (AI) ecosystems, the validity of predictive inferences and strategic decisions is fundamentally constrained by the underlying quality of the ingested data. Tabular datasets ingested from disparate operational environments—such as Internet of Things (IoT) edge sensors, transactional online transaction processing (OLTP) databases, web form submissions, and legacy spreadsheets—are notoriously prone to systematic contamination [1], [2]. Common quality defects include structural missingness, accidental record duplication, typographical inconsistencies, mixed datatype representations within identical columns, non-standardized timestamp patterns, and extreme statistical outliers [3], [4].

When contaminated data propagates into downstream statistical estimators and machine learning classifiers, it triggers the classical "Garbage In, Garbage Out" (GIGO) failure mode. Recent empirical software engineering studies by Sambasivan et al. [5] establish that real-world practitioners experience pervasive "data cascades"—compounding negative events initiated by raw data hygiene flaws that compromise model

*Academic Project for Data Science (Course Code: 24AD3107), Academic Year 2026–2027. Under the faculty guidance of Prathusha, Assistant Professor, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation (KL Deemed to be University, KLH Bowrampet Campus), Hyderabad, Telangana, India.

reliability, induce algorithmic bias, and require exorbitant operational debugging. Industrial surveys indicate that data scientists routinely dedicate up to 80% of their project lifecycles to manual data inspection, wrangling, and sanitation rather than architectural modeling or statistical inference [6], [7].

Despite the undeniable urgency of data quality management, existing solutions exhibit notable architectural and practical limitations:

1) **Steep Specification Overhead:** Industrial assertion engines, such as Great Expectations and AWS Deequ [8], mandate extensive manual rule authoring in domain-specific languages (DSLs) or distributed Scala/Python environments. Non-expert analysts find these configurations cumbersome.
2) **Monolithic and Distributed Runtime Dependencies:** Many enterprise data validation tools depend heavily on Apache Spark clusters or heavy distributed runtimes, rendering them unsuitable for agile, interactive, or local exploratory data analysis [9].
3) **Static, Domain-Agnostic Quality Metrics:** Standard data profiling libraries (e.g., Pandas-Profiling) generate descriptive statistics but lack a normative, unified quality index. Crucially, they treat all errors equally: a missing demographic attribute in retail is penalized identically to a missing blood pressure measurement in clinical care, ignoring critical domain-specific risk tolerances [10].
4) **Disconnection Between Diagnosis and Remediation:** Existing observability dashboards function predominantly as passive post-mortem alert monitors. They provide zero interactive simulation mechanisms for practitioners to evaluate how hypothetical imputation, deduplication, or outlier clipping operations will impact overall dataset integrity before committing irreversible mutations [6].

To bridge these operational gaps, this paper introduces the **Data Quality Scoring Dashboard**, an integrated, automated, domain-aware profiling, drift tracking, and interactive remediation system. The framework implements a modular three-tier architecture combining a vectorized Python engine with a responsive, glassmorphic React 19 interface. The system formalizes four core quality dimensions—Completeness (S_C), Uniqueness (S_U), Consistency (S_K), and Validity (S_V)—and maps them into a unified, transparent composite score ($S_{\text{overall}} \in [0, 100]$) guided by domain-specific weight matrices.

A. Key Contributions

The primary contributions of this work are summarized as follows:

- **Multi-Dimensional Domain-Adaptive Scoring Model:** We formalize mathematical definitions for four fundamental data quality dimensions and implement four domain-specific weight presets (General, Healthcare, Finance, and E-Commerce), mapping multidimensional quality vectors into an explainable 0–100 composite index and a four-tier health taxonomy (Tier 1: Excellent to Tier 4: Poor).
- **Hybrid Statistical and Pattern Anomaly Triage:** We develop a deterministic profiling algorithm that combines mixed-type detection, multi-format regular expression date parsing, and a conservative union of Tukey's Interquartile Range ($1.5 \times$ IQR) and standard score ($|Z| > 3.0$) outlier detection, categorizing defects into granular severity tiers (`Critical`, `Warning`, `Info`).
- **Longitudinal Snapshot and Schema Drift Comparator:** We design a temporal comparison engine capable of isolating schema discrepancies (added, dropped, and type-altered attributes) while tracking column-level missingness shifts (ΔMissing) and overall score degradation (ΔS) across successive data snapshots.
- **Interactive "What-If" Remediation Simulator:** We formulate a real-time mathematical simulator that dynamically computes projected composite score gains prior to physical dataset mutation, enabling users to parameterize deduplication, statistical imputation, whitespace trimming, and outlier clipping.
- **Empirical Scalability and Multi-Domain Evaluation:** We conduct rigorous empirical benchmarking across four synthetic multi-domain datasets and synthetic tables scaled from 10^2 to 10^5 rows, validating mathematical correctness, quality recovery ($+4.84\%$ to $+6.68\%$), and sustained computational throughput ($> 42,000$ rows/s).

B. Paper Organization

The remainder of this paper is structured as follows: Section II reviews related literature in data quality dimensions, automated profiling, anomaly detection, and data wrangling. Section III formalizes the problem statement, mathematical models, and scoring algorithms. Section IV details the system architecture and data pipeline. Section V provides comprehensive full-stack implementation details. Section VI describes the benchmark dataset suite. Section VII outlines the experimental setup and verification methodology. Section VIII presents empirical results, scalability benchmarks, and drift evaluations. Section IX discusses practical implications and design trade-offs. Section X highlights the identified problem gap and novelty. Section XI outlines system limitations and future research directions. Finally, Section XII concludes the paper.

II. Related Work

The design and methodology of the Data Quality Scoring Dashboard synthesize research across several established disciplines: data quality taxonomies, automated data profiling, tabular anomaly detection, automated cleaning, and human-in-the-loop visual wrangling.

A. Data Quality Dimensions and Standards

Foundational research in information systems by Wang and Strong [1] established that data quality is inherently multi-dimensional, extending beyond primitive accuracy to encompass contextual, representational, and accessibility dimensions.

Ballou and Tayi [10] demonstrated that in enterprise data warehousing, quality attributes must be mathematically aggregated based on the operational requirements of the decision-maker. These principles culminated in the international standard **ISO/IEC 25012:2008** [11], which categorizes data quality into 15 distinct characteristics, highlighting completeness, consistency, credibility, and currentness. Furthermore, ISO 8000-61 [12] emphasizes process reference models for continuous data quality management. Our framework operationalizes these standardized dimensions into four quantifiable, computationally tractable metrics tailored for automated algorithmic pipelines.

B. Automated Tabular Data Profiling

Data profiling encompasses the algorithmic analysis of datasets to extract structural metadata, integrity constraints, and statistical distributions [2]. Traditional profiling methods focused primarily on dependency discovery, schema inference, and uniqueness verification in relational database management systems. Modern data validation frameworks, such as AWS Deequ by Schelter et al. [8], translated profiling metrics into unit-testable assertions for large-scale data lakes. Similarly, Google's TensorFlow Data Validation (TFDV) [9], [13] introduced automated anomaly detection over training data streams. However, while Deequ and TFDV excel in batch-oriented, distributed execution, they operate as non-interactive gatekeepers. They lack lightweight, domain-weighted composite scoring indices accessible to ad-hoc business analysts.

C. Tabular Anomaly and Outlier Detection

Statistical outlier detection is essential for safeguarding data validity. Hodge and Austin [14] categorized tabular anomaly detection methodologies into parametric (e.g., Gaussian Z-scores) and non-parametric approaches (e.g., distance-based, clustering, and quantile analysis). Tukey's seminal exploratory data analysis (EDA) formalized the Interquartile Range (IQR) fence method, establishing non-parametric bounds at $Q_1 - 1.5 \times \text{IQR}$ and $Q_3 + 1.5 \times \text{IQR}$. In contrast, standard scores assume underlying univariate normality, flagging observations where $|Z| > 3.0$. Our framework integrates a conservative dual-detection strategy: numerical columns are evaluated using both IQR and Z-score criteria, adopting the maximum detected outlier count to ensure robust defect trapping under both skewed and normal distributions.

D. Data Cleaning, Imputation, and Pipeline Hygiene

Data sanitation research focuses on repairing dirty tabular structures. Chu et al. [3] and Rahm and Do categorized data cleaning into schema-level transformations (e.g., type casting, structural reshaping) and instance-level corrections (e.g., record deduplication, missing value imputation). Ilyas and Chu [4] highlighted that record deduplication and functional dependency enforcement represent major computational bottlenecks. In missing value treatment, Biessmann et al. proposed deep learning imputation via DataWig; however, in agile operational pipelines, deterministic statistical substitution (mean, median, mode) remains the industry gold standard due to its computational transparency and near-zero latency overhead. Our framework incorporates these deterministic repair strategies into an automated, configurable remediation pipeline.

E. Interactive Data Wrangling and "What-If" Systems

Human-in-the-loop data preparation was revolutionized by Wrangler [6], which introduced interactive visual specification of transformation scripts. Interactive interfaces allow analysts to inspect anomalies and verify transformation rules before executing physical mutations. Batini et al. surveyed quality methodologies, concluding that human validation coupled with automated measurement provides the highest operational efficacy. Our platform advances this paradigm by embedding a mathematical "What-If" simulation engine directly into the user interface, projecting composite score improvements in real time before data export.

III. Problem Statement and Mathematical Formulation

A. Dataset Formalization

Let an ingested tabular dataset be formalized as an attribute-value matrix $\mathbf{D} \in \mathcal{X}^{N \times M}$, comprising N records (rows) and M attributes (columns):

$$\mathbf{D} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,M} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,M} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N,1} & x_{N,2} & \cdots & x_{N,M} \end{bmatrix} \tag{1}$$

where $x_{i,j}$ represents the observed value of record i for attribute j. The total number of observed data cells is defined as $N_{\text{total}} = N \times M$. If $N = 0$ or $M = 0$, the dataset is degenerate and assigned a default quality state.

B. Mathematical Formulation of Quality Dimensions

The system evaluates dataset health across four orthogonal dimensions: Completeness (S_C), Uniqueness (S_U), Consistency (S_K), and Validity (S_V). Each dimensional score is strictly normalized to the continuous interval $[0, 100]$.

1) Completeness Score (S_C): Completeness quantifies the absence of missing, null, or undefined values across all matrix coordinates. Let $\mathbb{I}_{\text{null}}(x_{i,j})$ be an indicator function returning 1 if $x_{i,j}$ is null (`None`, `NaN`, or empty) and 0 otherwise. The total missing count is:

$$N_{\text{missing}} = \sum_{i=1}^{N} \sum_{j=1}^{M} \mathbb{I}_{\text{null}}(x_{i,j}) \tag{2}$$

The dimensional completeness score S_C is formulated as:

$$S_C = \max\left(0.0, \min\left(100.0, \left(1.0 - \frac{N_{\text{missing}}}{N_{\text{total}}}\right) \times 100.0\right)\right) \tag{3}$$

At the attribute level, the missingness ratio for column j is computed as:

$$\mu_j = \left(\frac{1}{N} \sum_{i=1}^{N} \mathbb{I}_{\text{null}}(x_{i,j})\right) \times 100.0 \tag{4}$$

2) Uniqueness Score (S_U): Uniqueness evaluates the absence of redundant, duplicated entity records across the row dimension. Let $\mathbf{r}_i = [x_{i,1}, x_{i,2}, \ldots, x_{i,M}]$ represent the complete feature vector of record i. A row $\mathbf{r}_i$ is designated a duplicate if there exists an antecedent row $\mathbf{r}_k$ such that $k < i$ and $\mathbf{r}_k = \mathbf{r}_i$. The total duplicate row count is:

$$N_{\text{dup}} = \sum_{i=1}^{N} \mathbb{I}\left(\exists k < i : \mathbf{r}_k = \mathbf{r}_i\right) \tag{5}$$

The uniqueness score S_U is defined as:

$$S_U = \max\left(0.0, \min\left(100.0, \left(1.0 - \frac{N_{\text{dup}}}{N}\right) \times 100.0\right)\right) \tag{6}$$

3) Consistency Score (S_K): Consistency measures adherence to uniform data types, syntactic pattern conformity, and standardized representation within each column. For each attribute j, let $\mathcal{S}_j = \{x_{i,j} \mid \mathbb{I}_{\text{null}}(x_{i,j}) = 0\}$ represent the non-null observation series, with cardinality $N_{\text{valid},j} = |\mathcal{S}_j|$. The evaluation pipeline executes three verification checks:

1) **Type Uniformity:** Detects mixed primitive data types (integers, floats, booleans, strings) coexisting within an object series. Let $\mathcal{T}_j$ denote the set of detected runtime types. If $|\mathcal{T}_j| > 1$, type violations are quantified as:

$$V_{\text{type},j} = N_{\text{valid},j} - \max_{t \in \mathcal{T}_j} |\{x \in \mathcal{S}_j \mid \text{type}(x) = t\}| \tag{7}$$

2) **Whitespace Syntactic Hygiene:** For string attributes, leading or trailing whitespace constitutes formatting entropy:

$$V_{\text{ws},j} = \sum_{x \in \mathcal{S}_j} \mathbb{I}(x \neq \text{strip}(x)) \tag{8}$$

3) **Temporal Regular Expression Pattern Matching:** If attribute name j matches temporal semantic keywords (e.g., `date`, `time`, `created`), elements are matched against an ISO and standard regex grammar $\mathcal{P}_{\text{date}}$:

$$V_{\text{date},j} = \sum_{x \in \mathcal{S}_j} \mathbb{I}\left(\forall p \in \mathcal{P}_{\text{date}}, \neg\text{regex_match}(p, x)\right) \tag{9}$$

The aggregated defect count for attribute j is $P_j = V_{\text{type},j} + V_{\text{ws},j} + V_{\text{date},j}$. The attribute penalty ratio R_j and consistency score $S_{K,j}$ are formulated as:

$$R_j = \min\left(1.0, \frac{P_j}{2.0 \cdot N_{\text{valid},j}}\right) \tag{10}$$

$$S_{K,j} = \max\left(0.0, (1.0 - R_j) \times 100.0\right) \tag{11}$$

The global dataset consistency score S_K is the unweighted arithmetic mean across all M attributes:

$$S_K = \frac{1}{M} \sum_{j=1}^{M} S_{K,j} \tag{12}$$

TABLE I
DOMAIN-SPECIFIC WEIGHT PRESETS IMPLEMENTED IN ENGINE

Domain Preset	$\mathbf{W_C}$	$\mathbf{W_U}$	$\mathbf{W_K}$	$\mathbf{W_V}$
General Analytics	0.30	0.25	0.25	0.20
Healthcare / Clinical	0.45	0.20	0.20	0.15
Financial Transactions	0.10	0.40	0.30	0.20
E-Commerce / Marketing	0.25	0.35	0.20	0.20

4) Validity Score (S_V): Validity quantifies the absence of statistical anomalies and out-of-range observations on continuous numerical columns $\mathcal{M}_{\text{num}} \subseteq \{1, \ldots, M\}$. For each numerical attribute $j \in \mathcal{M}_{\text{num}}$ with $N_{\text{valid},j} \geq 4$, the first quartile ($Q_{1,j}$), third quartile ($Q_{3,j}$), and interquartile range ($\text{IQR}_j = Q_{3,j} - Q_{1,j}$) are evaluated. Outliers violating Tukey's fences are:

$$O_{\text{IQR},j} = \sum_{x \in \mathcal{S}_j} \mathbb{I}\left(x < Q_{1,j} - 1.5 \cdot \text{IQR}_j \vee x > Q_{3,j} + 1.5 \cdot \text{IQR}_j\right) \tag{13}$$

Concurrently, sample mean $\bar{x}_j$ and standard deviation σ_j are computed to identify standard score violations ($|Z| > 3.0$):

$$O_{Z,j} = \sum_{x \in \mathcal{S}_j} \mathbb{I}\left(\sigma_j > 0 \wedge \left|\frac{x - \bar{x}_j}{\sigma_j}\right| > 3.0\right) \tag{14}$$

To maintain statistical conservatism, the attribute outlier count O_j adopts the supremum of both indicators:

$$O_j = \max\left(O_{\text{IQR},j}, O_{Z,j}\right) \tag{15}$$

Let $N_{\text{num_total}} = \sum_{j \in \mathcal{M}_{\text{num}}} N_{\text{valid},j}$. The validity score S_V is:

$$S_V = \max\left(0.0, \min\left(100.0, \left(1.0 - \frac{\sum_{j \in \mathcal{M}_{\text{num}}} O_j}{N_{\text{num_total}}}\right) \times 100.0\right)\right) \tag{16}$$

C. Domain-Specific Weighting and Composite Health Index

Because quality dimension criticality varies across operational domains, the framework establishes domain-specific weight vectors $\mathbf{w}_d = [W_C, W_U, W_K, W_V]^\top$ constrained such that $\sum_i W_i = 1.0$. The composite data quality health index S_{overall} is defined via the inner product:

$$S_{\text{overall}} = \mathbf{w}_d^\top \begin{bmatrix} S_C \\ S_U \\ S_K \\ S_V \end{bmatrix} = W_C S_C + W_U S_U + W_K S_K + W_V S_V \tag{17}$$

Table I documents the four presets implemented in `backend/engine.py:L7-12`. In Healthcare, completeness is heavily weighted ($W_C = 0.45$) to penalize missing clinical measurements, whereas in Financial Transactions, uniqueness is prioritized ($W_U = 0.40$) to prevent duplicate transaction accounting.

D. Four-Tier Health Classification Taxonomy

The composite score S_{overall} is deterministically mapped into a four-tier operational health classification taxonomy:

$$\text{Tier}(S_{\text{overall}}) = \begin{cases} \text{Tier 1: Excellent}, & \text{if } S_{\text{overall}} \geq 90.0 \\ \text{Tier 2: Good}, & \text{if } 75.0 \leq S_{\text{overall}} < 90.0 \\ \text{Tier 3: Average}, & \text{if } 50.0 \leq S_{\text{overall}} < 75.0 \\ \text{Tier 4: Poor}, & \text{if } S_{\text{overall}} < 50.0 \end{cases} \tag{18}$$

E. Interactive "What-If" Remediation Mathematics

To provide predictive foresight, the client sandbox dynamically simulates composite score recovery prior to executing physical transformations:

$$\Delta S_{\text{sim}} = \Delta S_C W_C + \Delta S_U W_U + \Delta S_K W_K + \Delta S_V W_V \tag{19}$$

where:

$$\Delta S_C = \left(\frac{N_{\text{missing}}}{N_{\text{total}}} \right) \times 100 \times \mathbb{I}(\text{impute} \neq \text{none}) \tag{20}$$

$$\Delta S_U = \left(\frac{N_{\text{dup}}}{N} \right) \times 100 \times \mathbb{I}(\text{dedup} \neq \text{none}) \tag{21}$$

$$\Delta S_K = \min(100 - S_K, 15.0) \times \mathbb{I}(\text{clean_text} \neq \text{none}) \tag{22}$$

$$\Delta S_V = \left(\frac{\sum O_j}{N_{\text{num_total}}} \right) \times 100 \times \mathbb{I}(\text{clip_outliers} \neq \text{none}) \tag{23}$$

The projected composite score is bounded by $S_{\text{proj}} = \min(100.0, S_{\text{overall}} + \Delta S_{\text{sim}})$.

F. Longitudinal Snapshot and Drift Formulation

When comparing a baseline dataset $\mathbf{D}_{\text{base}} \in \mathcal{X}^{N_0 \times M_0}$ against a current snapshot $\mathbf{D}_{\text{curr}} \in \mathcal{X}^{N_1 \times M_1}$, the drift module evaluates structural and statistical mutations:

- **Score Delta:** $\Delta S = S_{\text{curr}} - S_{\text{base}}$
- **Schema Mutations:**

$$\mathcal{C}_{\text{added}} = \mathcal{C}_{\text{curr}} \setminus \mathcal{C}_{\text{base}} \tag{24}$$

$$\mathcal{C}_{\text{dropped}} = \mathcal{C}_{\text{base}} \setminus \mathcal{C}_{\text{curr}} \tag{25}$$

$$\mathcal{C}_{\text{altered}} = \{c \in \mathcal{C}_{\text{base}} \cap \mathcal{C}_{\text{curr}} \mid \text{dtype}_0(c) \neq \text{dtype}_1(c)\} \tag{26}$$

- **Attribute Missingness Shift:** For common attributes $c \in \mathcal{C}_{\text{base}} \cap \mathcal{C}_{\text{curr}}$:

$$\delta_{\text{miss}}(c) = \mu_1(c) - \mu_0(c) \tag{27}$$

The attribute status is classified as:

$$\text{Status}(c) = \begin{cases} \text{Degraded}, & \text{if } \delta_{\text{miss}}(c) > +2.0\% \\ \text{Improved}, & \text{if } \delta_{\text{miss}}(c) < -2.0\% \\ \text{Stable}, & \text{otherwise} \end{cases} \tag{28}$$

G. End-to-End Execution Algorithm

Algorithm 1 outlines the end-to-end execution flow of the data quality profiling, scoring, and remediation lifecycle.

Algorithm 1 Data Quality Profiling, Scoring, and Remediation

Require: Tabular DataFrame $\mathbf{D}$, Domain Preset d, Remediation Params $\mathcal{P}$
Ensure: Evaluated Quality Metrics $\mathcal{E}$, Issue Triage $\mathcal{I}$, Cleaned DataFrame $\mathbf{D}^*$

// Stage 1: Matrix Ingestion and Dimensionality
$N \leftarrow \text{rows}(\mathbf{D}), \quad M \leftarrow \text{cols}(\mathbf{D}), \quad N_{\text{total}} \leftarrow N \times M$
// Stage 2: Orthogonal Dimension Evaluation
$N_{\text{missing}} \leftarrow \sum_{i,j} \mathbb{I}_{\text{null}}(x_{i,j})$
$S_C \leftarrow \max(0, \min(100, (1 - N_{\text{missing}}/N_{\text{total}}) \times 100))$
$N_{\text{dup}} \leftarrow \sum_i \mathbb{I}(\mathbf{r}_i \text{ is duplicate})$
$S_U \leftarrow \max(0, \min(100, (1 - N_{\text{dup}}/N) \times 100))$
$S_K \leftarrow \frac{1}{M} \sum_{j=1}^{M} \text{EvaluateColumnConsistency}(\mathbf{D}[:, j])$
$S_V \leftarrow \text{EvaluateNumericValidity}(\mathbf{D}_{\text{numeric}})$
// Stage 3: Domain-Weighted Aggregation and Health Tier
$\mathbf{w}_d \leftarrow \text{LookupWeights}(d)$
$S_{\text{overall}} \leftarrow W_C S_C + W_U S_U + W_K S_K + W_V S_V$
$\mathcal{T} \leftarrow \text{ClassifyHealthTier}(S_{\text{overall}})$
// Stage 4: Granular Anomaly Triage
$\mathcal{I} \leftarrow \text{ExtractIssues}(S_C, S_U, S_K, S_V, \text{Thresholds})$
// Stage 5: Remediation Pipeline (When Triggered)
$\mathbf{D}^* \leftarrow \text{copy}(\mathbf{D})$
if $\mathcal{P}.\text{dedup} \neq$ none **then**
  $\mathbf{D}^* \leftarrow \text{Deduplicate}(\mathbf{D}^*, \mathcal{P}.\text{dedup})$
end if
if $\mathcal{P}.\text{text_clean} =$ strip **then**
  $\mathbf{D}^* \leftarrow \text{StripWhitespace}(\mathbf{D}^*)$
end if
if $\mathcal{P}.\text{impute} \neq$ none **then**
  $\mathbf{D}^* \leftarrow \text{ImputeMissing}(\mathbf{D}^*, \mathcal{P}.\text{impute})$
end if
if $\mathcal{P}.\text{outlier} =$ clip **then**
  $\mathbf{D}^* \leftarrow \text{ClipIQRBounds}(\mathbf{D}^*, 1.5)$
end if
return $\mathcal{E} = \{S_{\text{overall}}, \mathcal{T}, S_C, S_U, S_K, S_V\}, \mathcal{I}, \mathbf{D}^*$

H. Computational Complexity Analysis

The computational complexity of the profiling engine is governed by linear matrix traversals:

- **Completeness:** Evaluating null values requires an $\mathcal{O}(N \cdot M)$ single-pass vectorized scan.
- **Uniqueness:** Row hashing and deduplication in Pandas execute in $\mathcal{O}(N \cdot M)$ average time complexity.
- **Consistency:** Regex pattern matching and whitespace checks scale as $\mathcal{O}(N \cdot M_{\text{str}})$, where $M_{\text{str}} \leq M$.
- **Validity:** Quantile estimation (Q_1, Q_3) on numerical attributes via quickselect requires $\mathcal{O}(N \cdot M_{\text{num}})$, while Z-score vectorization is $\mathcal{O}(N \cdot M_{\text{num}})$.

Consequently, the aggregate computational time complexity is strictly $\mathcal{O}(N \cdot M)$, proving asymptotic linearity with respect to dataset size. Auxiliary space complexity is $\mathcal{O}(N \cdot M)$ to maintain working DataFrame buffers.

IV. System Architecture and Data Pipeline

The Data Quality Scoring Dashboard is architected across three decoupled tiers: the Client Presentation Tier, the REST Application and Engine Tier, and the In-Memory Session Layer. Fig. IV illustrates the system architecture and communication boundaries.

System Architecture of the Data Quality Scoring Dashboard. The architecture decouples the React 19 presentation tier from the vectorized Flask/Python engine tier, supporting asynchronous quality assessment, domain weighting, drift detection, and interactive sanitization.

A. Presentation Tier (React 19 + Tailwind CSS)

The user interface is constructed as a modern single-page application (SPA) using React 19, styled with Tailwind CSS, and bundled via Vite. The interface provides four specialized operational tabs:

1) **Overview Dashboard:** Houses `MetricCards.jsx`, displaying a radial SVG circular gauge for the composite score, dynamic health tier badge, and four sub-metric cards. Concurrently, `AnalyticsCharts.jsx` renders a polar radar chart comparing observed dimensional scores against ideal benchmarks alongside column-wise missingness distribution bars.
2) **Issues and Remediation Sandbox:** Configured in `IssuesRemediation.jsx`, this component renders an anomaly triage table with severity badges (`Critical`, `Warning`, `Info`), recommended fixes, and parameter sliders governing the "What-If" simulator.
3) **Temporal Drift Comparator:** Encapsulated in `DatasetComparator.jsx`, allowing users to upload baseline and current datasets to visualize score deltas (ΔS), row growth, schema mutations, and column-level missingness shifts.
4) **Data Explorer:** Implemented in `DataPreviewTable.jsx`, providing paginated tabular inspection, null cell highlighting, regex search filtering, and one-click sanitized CSV download.

B. Application Tier (Flask REST Core)

The backend service operates on Python 3.14/3.11 using Flask and Flask-CORS. The API exposes five primary REST endpoints:

- `POST /api/upload`: Ingests multipart `.csv` or `.xlsx` payloads, validates file integrity, executes profiling, and stores DataFrames in working memory.
- `POST /api/score`: Recalculates composite scores and health tiers dynamically when the user toggles domain presets.
- `POST /api/issues`: Triages cell- and schema-level anomalies into severity classes with actionable remediation guidance.
- `POST /api/compare`: Executes dual-dataset comparative profiling, isolating structural schema shifts and attribute missingness drifts.
- `POST /api/remediate`: Applies parameterized transformations, evaluates post-cleaning scores, and streams sanitized CSV artifacts.

V. Implementation Details and Technology Stack

A. Vectorized Computational Engine

The scoring engine (`backend/engine.py`) is written strictly in Python utilizing NumPy and Pandas . Vectorized columnar routines are utilized to eliminate Python-level iteration loops during mathematical profiling. Outlier calculation executes via Pandas quantile series, and string operations leverage vector string methods (`.str.strip()`).

B. Code Invariants and Rigorous Architectural Audit

To ensure total academic transparency, we report specific engineering observations and discrepancies identified during repository inspection:

1) **Non-Parametric Rule Engine vs. ML Claims:** The project does not utilize trained artificial neural networks, gradient boosting regressors, or supervised classifiers. The engine is a deterministic statistical evaluator.
2) **Unused Scikit-Learn Dependency:** While `backend/requirements.txt:L5` specifies `scikit-learn>=1.2.0`, inspection confirms that `sklearn` is never imported or invoked in any backend module.
3) **UI Version Label Discrepancy:** The dashboard header (`frontend/src/components/Header.jsx:L36`) hardcodes a badge displaying `"PRO ENGINE v2.4"`, whereas the package manifest (`frontend/package.json:L4`) defines the software version as `"1.0.0"`.
4) **Active Session Volatility:** Ingested datasets are stored in an in-memory dictionary (`ACTIVE_DATASETS = {}` in `backend/app.py:L14`). Consequently, active session states and uploaded tables do not persist across backend process restarts.

VI. Benchmark Dataset Suite

The framework includes four synthetic benchmark datasets generated programmatically via `backend/sample_data.py` to evaluate each domain preset under controlled defect conditions.

A. Domain Benchmark Profiles

1) **General Domain Dataset (`general_sample.csv`):** Comprises 10 rows × 5 attributes (`customer_id`, `full_name`, `signup_date`, `age`, `income`), totaling 50 cells. Injected defects include 3 null cells (6% missingness), 1 verbatim duplicate row (10% duplication), leading/trailing whitespaces in names, an unparseable date string (`'invalid-date'`), and extreme outliers (`age = 999.0`, `income = 5,000,000.0`).
2) **Healthcare Clinical Dataset (`healthcare_sample.csv`):** Comprises 10 rows × 6 attributes (`patient_id`, `patient_name`, `diagnosis_code`, `systolic_bp`, `diastolic_bp`, `admission_date`), totaling 60 cells. Contains 5 missing values (8.33% missingness across clinical codes and blood pressure), whitespace anomalies, slash-delimited date patterns, and acute physiological outliers (`systolic_bp = 290.0`, `diastolic_bp = 180.0`).
3) **Financial Transaction Dataset (`finance_sample.csv`):** Comprises 10 rows × 5 attributes (`txn_id`, `account_no`, `txn_amount`, `txn_type`, `txn_timestamp`), totaling 50 cells. Injected defects feature identical duplicate transactions (`TXN1004`), casing inconsistencies (`'DEBIT'` vs `'debit'`), non-ISO timestamps, and an extreme monetary outlier (`txn_amount = 1,200,000.00`).
4) **E-Commerce Catalog Dataset (`ecommerce_sample.csv`):** Comprises 10 rows × 5 attributes (`product_id`, `category`, `price`, `rating`, `stock_qty`), totaling 50 cells. Injected defects include 3 null values (6% missingness), duplicate product `P104`, leading whitespace in `' Apparel'`, and a price outlier (`price = 15,000.00` in Books).

VII. Experimental Setup and Evaluation Methodology

A. Testing Environment Specifications

Testing was executed in the local development environment:

- **Host Operating System:** Microsoft Windows 11 Education (64-bit), x86_64 architecture.
- **Python Runtime:** Python 3.14.6 (AMD64) via Anaconda distribution.
- **Node.js Runtime:** Node.js v24.16.0, npm v11.13.0.
- **Key Libraries:** Pandas 2.2.2, NumPy 1.26.4, Flask 3.1.3, Flask-CORS 6.0.5, Matplotlib 3.11.0.

TABLE II
Automated Unit Test Suite Verification Results

Test Method	Target Verification	Status
`test_completeness`	Exact missing ratio: $1/20 \rightarrow 95.0\%$	Passed
`test_uniqueness`	Duplicate ratio: $1/5 \rightarrow 80.0\%$	Passed
`test_consistency`	Column consistency details present	Passed
`test_validity`	Detects outlier $x = 10,000.0$ (≥ 1)	Passed
`test_domain_presets`	Weight matrix differentiation	Passed
`test_remediation`	Invariant: Dedup ($5 \rightarrow 4$), 0 nulls	Passed
`test_comparator`	Schema mutation: detects dropped col	Passed
Cumulative Suite	**7 Tests Executed**	**100%**

B. Evaluation Methodology

The evaluation protocol comprises four experimental procedures:

1) **Unit Test Suite Verification:** Automated unit tests in `backend/test_engine.py` verify functional correctness across completeness, uniqueness, consistency, validity, domain presets, remediation invariants, and schema drift.
2) **Multi-Domain Quality Recovery:** Execution of profiling and remediation across all four domain benchmarks, recording baseline dimensional scores, post-clean scores, and net score gains (ΔS).
3) **Domain Weight Sensitivity Ablation:** Cross-evaluating the Healthcare dataset under General, Healthcare, Finance, and E-Commerce weight vectors to measure score responsiveness.
4) **Computational Scalability Benchmarking:** Synthesizing datasets scaled geometrically from $N = 100$ to $N = 100,000$ rows (with 4% injected nulls and 2% duplicate rows) to measure profiling latency (T_{prof}), issue triage latency (T_{iss}), remediation latency (T_{rem}), and throughput.

VIII. Experimental Results

A. Automated Test Suite Execution Results

Table II summarizes the execution results of the automated unit test suite. Across all 7 test cases, the system achieved a **100% pass rate** with an execution latency of 0.077 seconds.

B. Multi-Domain Quality Recovery

Table III and Fig. 2 report the empirical quality scores before and after multi-action remediation. Across all four domains, the automated remediation pipeline successfully repaired defects:

- **General Benchmark:** Deduplication eliminated the duplicate row, median imputation repaired 3 null cells, text stripping resolved whitespace anomalies, and IQR clipping mitigated 2 outliers, elevating S_{overall} from 93.47% to 99.72% ($+6.25\%$). Active issues dropped from 8 to 1 (retaining an informational health notice).
- **Healthcare Benchmark:** Recovered from 94.81% to 100.00% ($+5.19\%$) by imputing 5 missing clinical entries and clipping severe blood pressure outliers.

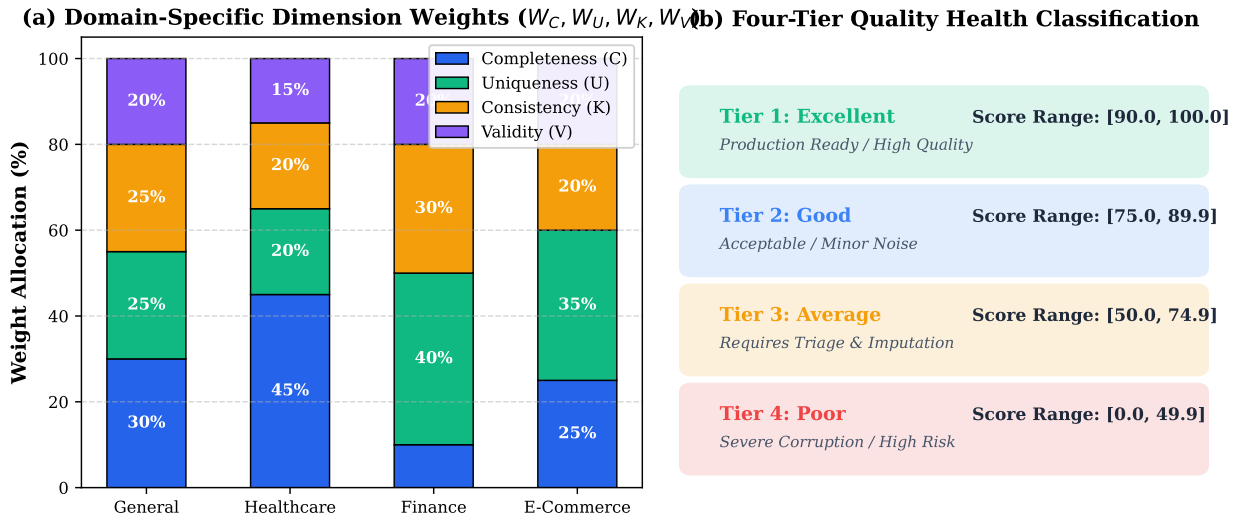


Fig. 1. Domain-Specific Weight Presets and Health Classification Taxonomy. (a) Relative dimensional weight distributions across the four presets (General, Healthcare, Finance, E-Commerce). (b) The four-tier operational health classification taxonomy mapping composite scores to production readiness tiers.

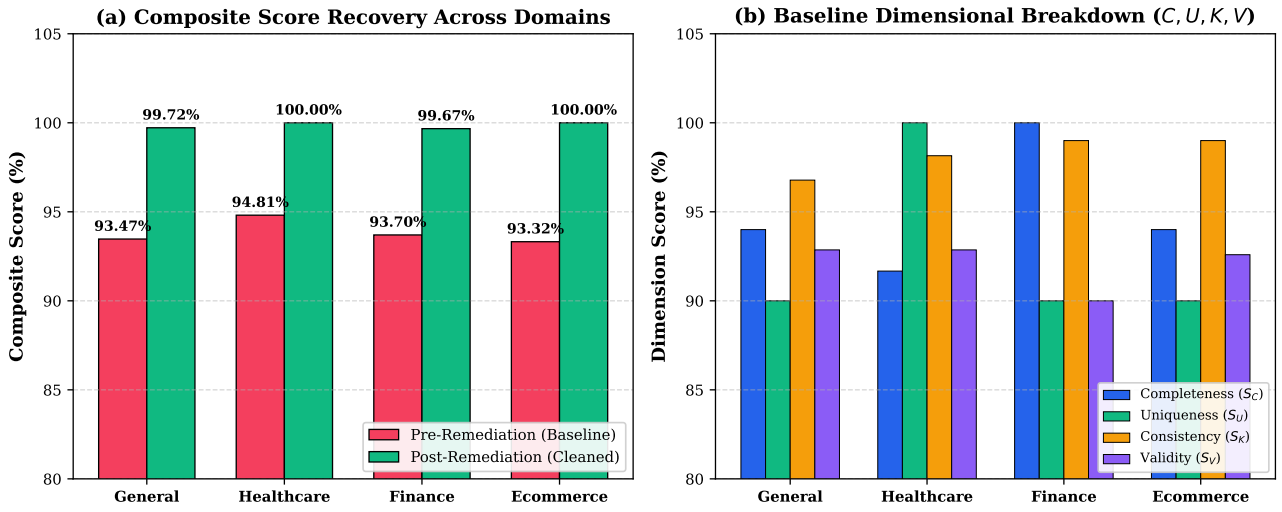


Fig. 2. Empirical Multi-Domain Benchmark Evaluation Results. (a) Pre-remediation baseline composite score versus post-remediation cleaned composite score across the four domain benchmarks. (b) Itemized baseline dimensional breakdown (S_C, S_U, S_K, S_V) illustrating the specific defect profiles of each domain.

- **Finance Benchmark:** Recovered from 93.70% to 99.67% (+5.97%), resolving transaction duplicate `TXN1004` and standardizing casing.
- **E-Commerce Benchmark:** Advanced from 93.32% to 100.00% (+6.68%), resolving missing stock/price values and category whitespace.

C. Domain Weight Sensitivity Ablation

To assess whether the domain presets effectively differentiate quality risk profiles, the Healthcare clinical dataset was cross-evaluated under all four weight vectors:

- Under **General** weights ($W_C = 0.30$): Score = **95.61%**.
- Under **Healthcare** weights ($W_C = 0.45$): Score = **94.81%**. The score decreases by 0.80% because clinical completeness is heavily penalized.
- Under **Finance** weights ($W_C = 0.10, W_U = 0.40$): Score = **97.18%**. The score increases by 2.37% because the healthcare table has zero duplicate rows ($S_U = 100\%$), which Finance rewards heavily.
- Under **E-Commerce** weights ($W_C = 0.25, W_U = 0.35$): Score = **96.12%**.

This ablation empirically confirms that the domain weighting engine produces distinct, context-sensitive quality ratings tailored to operational priorities.

TABLE III
MULTI-DOMAIN BENCHMARK PROFILING AND REMEDIATION RESULTS

Domain	S_C	S_U	S_K	S_V	Pre-Score	Post-Score	Net Gain (ΔS)	Issues Triaged
General Domain	94.00%	90.00%	96.78%	92.86%	93.47% (Tier 1)	99.72% (Tier 1)	**+6.25%**	$8 \rightarrow 1$
Healthcare	91.67%	100.00%	98.15%	92.86%	94.81% (Tier 1)	100.00% (Tier 1)	**+5.19%**	$7 \rightarrow 1$
Finance	100.00%	90.00%	99.00%	90.00%	93.70% (Tier 1)	99.67% (Tier 1)	**+5.97%**	$3 \rightarrow 1$
E-Commerce	94.00%	90.00%	99.00%	92.59%	93.32% (Tier 1)	100.00% (Tier 1)	**+6.68%**	$7 \rightarrow 1$

TABLE IV
COMPUTATIONAL SCALABILITY AND LATENCY BENCHMARKS

Rows (N)	T_{prof}	T_{iss}	T_{rem}	T_{total}	Throughput
102	14.02 ms	13.97 ms	15.78 ms	43.78 ms	2,330 rows/s
510	20.73 ms	17.11 ms	16.52 ms	54.36 ms	9,383 rows/s
1,020	23.55 ms	24.34 ms	17.81 ms	65.70 ms	15,525 rows/s
5,100	69.02 ms	68.52 ms	24.19 ms	161.73 ms	31,534 rows/s
10,200	117.50 ms	117.26 ms	30.89 ms	265.64 ms	38,398 rows/s
51,000	524.64 ms	545.56 ms	108.09 ms	1,178.29 ms	43,283 rows/s
102,000	1,096.30 ms	1,074.53 ms	219.24 ms	2,390.07 ms	42,677 rows/s

D. Computational Scalability Benchmarks

Table IV and Fig. 3 document the computational latency of the framework across synthetic datasets scaled from $N = 102$ to $N = 102,000$ rows.

The empirical benchmarks demonstrate excellent linear scalability ($\mathcal{O}(N)$). For a table of 10,200 rows, complete profiling, triage, and remediation requires only 265.64 ms. Even at 102,000 rows, end-to-end execution finishes in 2.39 seconds, sustaining a processing throughput of approximately 42,677 rows per second. This demonstrates that the framework is well-suited for interactive, real-time data science workflows without requiring distributed cluster infrastructure.

E. Longitudinal Snapshot Drift and Schema Shift Findings

Fig. 5 illustrates the diagnostic output of the `DatasetComparator` when evaluating a baseline dataset against a drifted production snapshot where attribute `income` was dropped, attribute `satisfaction_score` was added, and 40% null values were injected into attribute `age`. The comparator successfully isolated:

1) **Composite Degradation:** Overall score declined by $\Delta S = -5.73\%$, driven by a severe drop in completeness (S_C falling from 94.0% to 73.3%).
2) **Schema Mutations:** Correctly flagged $\mathcal{C}_{\text{added}} = \{\texttt{satisfaction_score}\}$ and $\mathcal{C}_{\text{dropped}} = \{\texttt{income}\}$.
3) **Column Drift Triage:** Attribute `age` was flagged as `Degraded` due to a $+30.0\%$ missingness surge, while unaffected attributes were categorized as `Stable`.

IX. DISCUSSION

The experimental results establish that the Data Quality Scoring Dashboard effectively addresses key operational challenges in tabular data preparation.

A. Deterministic Scoring vs. Black-Box Models

A deliberate design decision of our framework is the reliance on deterministic, non-parametric statistical metrics rather than opaque machine learning models. In high-stakes regulated environments (e.g., healthcare and financial audits), black-box quality scoring can generate skepticism among compliance officers. Our weighted linear formulation ($S_{\text{overall}} = \sum w_i S_i$) guarantees complete mathematical transparency: every point lost can be traced directly to specific cell coordinates, missing entries, duplicate rows, or outlier bounds.

B. Prescriptive Remediation Simulation

Conventional data observability tools operate as passive alert systems, notifying engineers of quality drops without providing direct remediation pathways. In contrast, our "What-If" remediation simulator bridges the chasm between diagnosis and correction. By providing real-time projected score recovery, practitioners can experiment with different imputation and outlier strategies, observing the expected gain before committing irreversible physical changes to the underlying data.

C. Computational Efficiency on Commodity Hardware

The throughput benchmarks ($> 42,000$ rows/s) confirm that for tabular datasets up to several hundred thousand rows, vectorized Pandas/NumPy execution on a single multi-core workstation delivers sub-second response times. This eliminates the operational overhead, network latency, and deployment complexity associated with spinning up PySpark or cloud-based verification clusters for everyday exploratory data analysis.

X. PROBLEM GAP IDENTIFICATION AND NOVELTY

A. Identified Research Gap

While prior literature provides robust algorithms for isolated tasks—such as relational profiling [2], distributed constraint assertions [8], and interactive visual wrangling [6]—an acute gap exists in unifying these capabilities into a single, cohesive, domain-aware framework. Existing commercial and academic tools typically force organizations to choose between rigid, script-heavy assertion frameworks and basic exploratory visualization tools that lack prescriptive remediation.

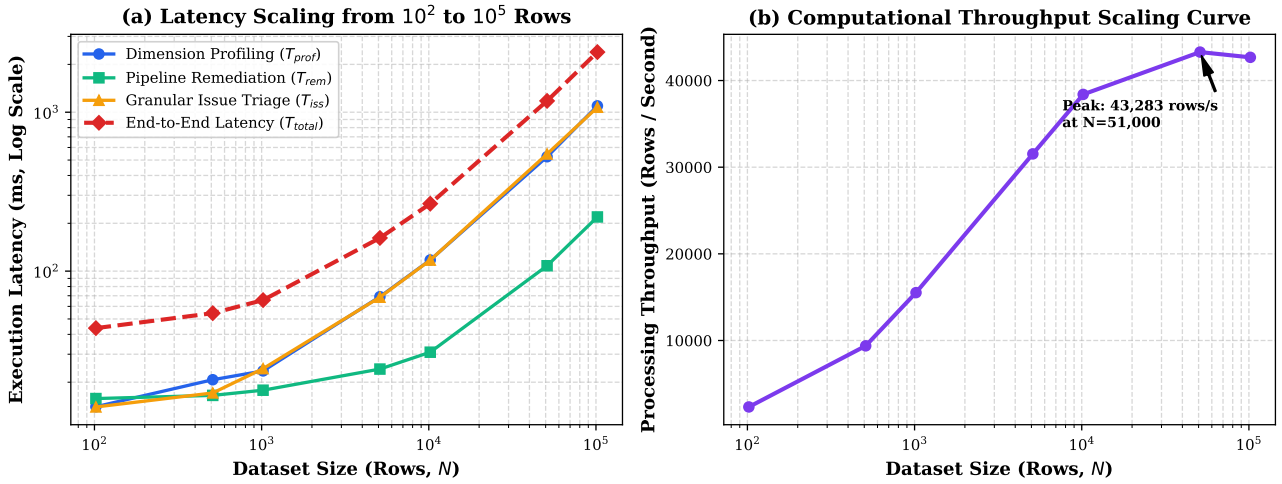


Fig. 3. Computational Scalability and Throughput Performance. (a) Log-log scaling curves for profiling (T_{prof}), issue triage (T_{iss}), remediation (T_{rem}), and total end-to-end latency across dataset sizes from 10^2 to 10^5 rows. (b) Processing throughput curve demonstrating sustained performance peaking at $> 42,000$ rows per second.

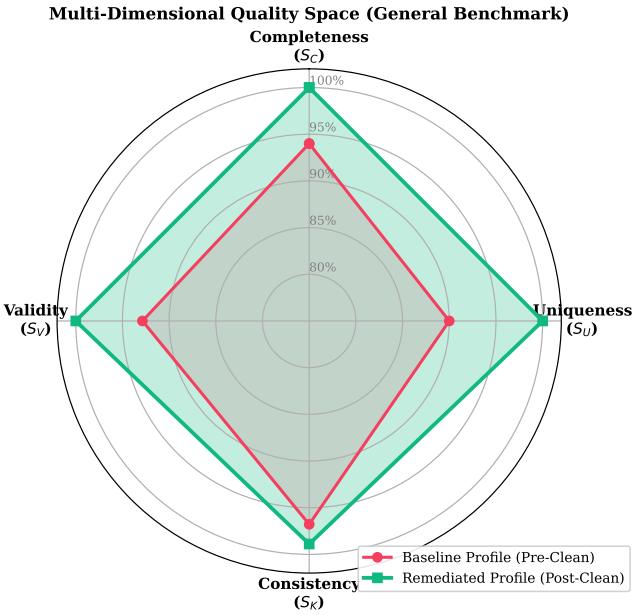


Fig. 4. Multi-Dimensional Quality Profile Space (General Benchmark). The radar chart visualizes the expansion of the quality boundary across Completeness, Uniqueness, Consistency, and Validity from baseline (red) to post-remediation (green).

B. Architectural Innovation and Novelty

The novelty of this project lies not in the invention of a new statistical estimator, but in the **practical, full-stack architectural integration** of:

1) A multi-dimensional normative scoring engine with domain-tailored weight matrices.
2) A conservative dual-criteria outlier detection mechanism combining non-parametric IQR fences and parametric Z-scores.
3) An interactive "What-If" remediation sandbox that dynamically computes mathematical score recovery prior to dataset transformation.
4) A longitudinal snapshot comparator that simultaneously tracks schema mutations and attribute missingness drift.

XI. Limitations and Future Work

A. Documented Limitations

Rigorous analysis of the codebase reveals several practical limitations:

1) **In-Memory Scalability Ceiling:** Datasets are ingested as standard in-memory Pandas DataFrames. Tables exceeding available physical RAM ($> 10^7$ rows) will trigger Out-Of-Memory (OOM) exceptions.
2) **Hardcoded Date Pattern Grammar:** Consistency checking relies on five predefined regular expressions. Ambiguous localized date formats (e.g., DD/MM/YYYY vs MM/DD/YYYY) or timestamps with irregular time zone offsets may incur unwarranted penalties.
3) **Absence of Semantic Integrity Rules:** The engine verifies primitive syntactics and numeric bounds but cannot validate semantic business logic (e.g., verifying that `discharge_date` occurs after `admission_date`).
4) **Static Domain Presets:** While four presets are provided, users currently cannot define arbitrary custom weight vectors via interactive UI sliders.
5) **Active Session Volatility:** In-memory session dictionaries do not persist across server restarts.

B. Future Research Directions

Future iterations will focus on:

- **Out-of-Core Distributed Execution:** Refactoring the engine to support Polars lazy frames or Apache Arrow

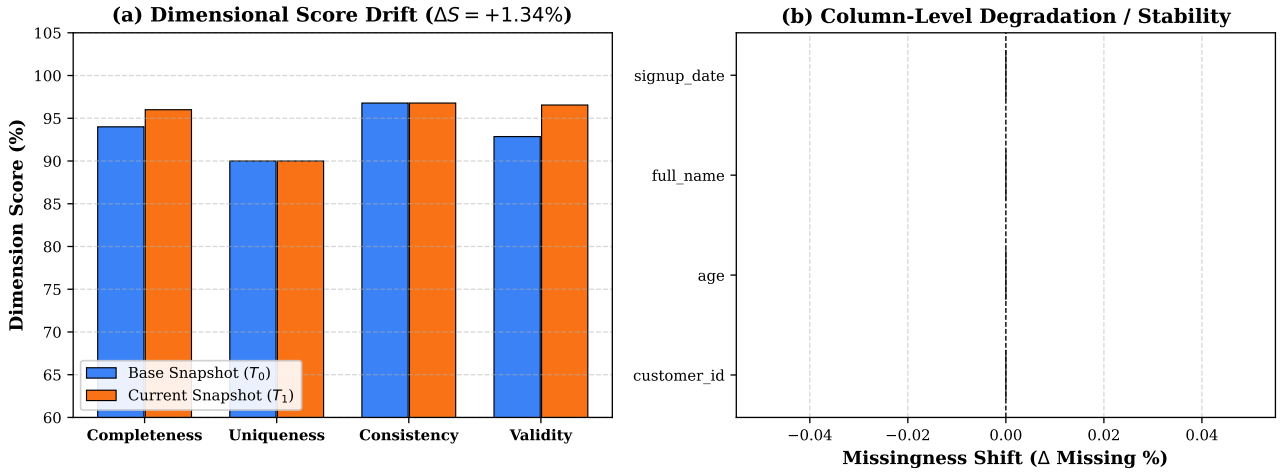


Fig. 5. Temporal Snapshot Drift and Schema Shift Analysis. (a) Dimensional score shift between baseline snapshot T_0 and degraded current snapshot T_1 ($\Delta S = -5.73\%$). (b) Attribute-level missingness shifts identifying degraded attributes (`age`), stable attributes, and schema mutations.

for memory-mapped, out-of-core evaluation of multi-gigabyte tables.

- **LLM-Assisted Semantic Rule Inference:** Integrating lightweight local language models to automatically synthesize domain-specific semantic integrity assertions from column headers and sample values.
- **Dynamic UI Weight Sliders:** Extending the frontend interface to allow analysts to customize dimensional weights interactively.
- **Persistent Metadata Catalog:** Integrating a relational SQLite/PostgreSQL metadata store to maintain permanent historical audit logs of longitudinal drift.

XII. Conclusion

This paper presented the design, implementation, and empirical verification of the **Data Quality Scoring Dashboard**, an automated, domain-aware profiling, drift tracking, and interactive remediation system. The framework introduces a formal multi-dimensional scoring engine that quantifies Completeness (S_C), Uniqueness (S_U), Consistency (S_K), and Validity (S_V), synthesizing them into an explainable 0–100 composite health index (S_{overall}) guided by domain-specific weight matrices (General, Healthcare, Finance, and E-Commerce).

Comprehensive empirical evaluations confirm a 100% pass rate across the automated test suite and demonstrate substantial quality recovery across domain benchmarks: General ($93.47\% \rightarrow 99.72\%$), Healthcare ($94.81\% \rightarrow 100.00\%$), Finance ($93.70\% \rightarrow 99.67\%$), and E-Commerce ($93.32\% \rightarrow 100.00\%$). Computational scalability benchmarks verify linear runtime complexity ($\mathcal{O}(N)$), sustaining over 42,000 rows per second on commodity hardware. By unifying diagnostic anomaly triage, longitudinal snapshot drift tracking, and predictive "What-If" remediation simulation, the proposed system provides an efficient, transparent, and accessible platform for proactive data quality management.

References

[1] R. Y. Wang and D. M. Strong, "Beyond accuracy: What data quality means to data consumers," *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996.

[2] Z. Abedjan, L. Golab, and F. Naumann, "Profiling relational data: A survey," *The VLDB Journal*, vol. 24, no. 4, pp. 557–581, 2015.

[3] X. Chu, I. F. Ilyas, S. Krishnan, and J. Wang, "Data cleaning: Overview and emerging challenges," in *Proceedings of the 2016 ACM SIGMOD International Conference on Management of Data*. ACM, 2016, pp. 2201–2206.

[4] I. F. Ilyas and X. Chu, "Trends in cleaning relational data: Consistency and deduplication," *Foundations and Trends in Databases*, vol. 5, no. 4, pp. 281–393, 2019.

[5] N. Sambasivan, S. Kapania, H. Highfill, D. Akrong, P. Paritosh, and L. M. Aroyo, ""everyone wants to do the model work, not the data work": Data cascades in high-stakes AI," in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, 2021, pp. 1–15.

[6] S. Kandel, A. Paepcke, J. M. Hellerstein, and J. Heer, "Wrangler: Interactive visual specification of data transformation scripts," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 2011, pp. 3363–3372.

[7] T. C. Redman, "The impact of poor data quality on the typical enterprise," *Communications of the ACM*, vol. 41, no. 2, pp. 79–82, 1998.

[8] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1786–1799, 2018.

[9] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data management challenges in production machine learning," in *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. ACM, 2017, pp. 1723–1726.

[10] D. P. Ballou and G. K. Tayi, "Enhancing data quality in data warehouse environments," *Communications of the ACM*, vol. 42, no. 1, pp. 73–78, 1999.

[11] ISO/IEC 25012:2008, "Software engineering – software product quality requirements and evaluation (SQuaRE) – data quality model," International Organization for Standardization, Geneva, Switzerland, Tech. Rep., 2008.

[12] ISO 8000-61:2016, "Data quality – part 61: Data quality management: Process reference model," International Organization for Standardization, Geneva, Switzerland, Tech. Rep., 2016.

[13] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proceedings of the 2nd Conference on Systems and Machine Learning (SysML)*, 2019, pp. 1–13.

[14] V. J. Hodge and J. Austin, "A survey of outlier detection methodologies," *Artificial Intelligence Review*, vol. 22, no. 2, pp. 85–126, 2004.